

Deviation Management – Revision after design review

The deviation now stores all thirteen parameters plus the filtering law and the TE in its own table, instead of adding columns to THRESHOLD_CONFIGURATION. Part 1 undoes the persistence work from the previous steps. Part 2 builds the new one. Everything on the screen itself stays exactly as it is.

Keep – no change needed

File	What stays
WCModelDisplayProperty	Column widths, FILTERING_LAW and TE constants, CHECKBOX removed, hardened fromKey
WCTreeLabelProvider	getColumnImage returning null, the two getColumnText cases, green row in getBackground
WCTreeCellModifier	Deviated guard in canModify, the two cases in getValue and modify
WCModelTreeView	The ComboBoxViewerCellEditor for Filtering Law
WCThresholdModel	All five fields, plus isModifiedAgainstBaseline()
WCThresholdBaseline	Kept – still how Apply decides a row was changed
HealthDao	getFilteringLaws().size() in getLatestConfiguration
IHealthCommonBusiness HealthCommonBusiness	getFilteringLawNames
HealthAlarmsPreferenceStore	getFilteringLawNames, the baseline and law seeding in getThresholdModels, the TE check in isValidData
AircraftSelectionComposite	The centred TE guidance message
HealthAlarmsComposite	The three tabs
messages.properties	All keys

The screen behaves identically. Only the destination of the data changes.

Part 1 – Remove

Each step below undoes one step of the original document. The code shown is exactly what was added there, so delete precisely those lines and nothing else.

Step 1 — Undo original Step 18 – ThresholdConfiguration

mgs-jpa-model/src/main/java/com/eurocopter/mgs/datamodel/health/monitoring/ThresholdConfiguration.java

Delete these, which were added after the workcard field.

```

@Column(name = "TE_REFERENCE")
private String teReference;

@Temporal(TemporalType.TIMESTAMP)
@Column(name = "DEVIATION_START_DATE")
private Date deviationStartDate;

@Temporal(TemporalType.TIMESTAMP)
@Column(name = "DEVIATION_END_DATE")
private Date deviationEndDate;

public String getTeReference() { return teReference; }
public void setTeReference(String teReference) { this.teReference = teReference; }
public Date getDeviationStartDate() { return deviationStartDate; }
public void setDeviationStartDate(Date d) { this.deviationStartDate = d; }
public Date getDeviationEndDate() { return deviationEndDate; }
public void setDeviationEndDate(Date d) { this.deviationEndDate = d; }

public boolean isDeviationActive() {
    return deviationStartDate != null && deviationEndDate == null;
}

```

Then remove the now-unused imports: `java.util.Date`, `jakarta.persistence.Temporal` and `jakarta.persistence.TemporalType`. The class must end up identical to what it was before the BCR.

Step 2 — Undo original Step 19 – DDL / migration script

Delete these three statements. If the migration has already been applied to a database, add the matching drops instead.

```

// DELETE these
ALTER TABLE THRESHOLD_CONFIGURATION ADD TE_REFERENCE VARCHAR(255);
ALTER TABLE THRESHOLD_CONFIGURATION ADD DEVIATION_START_DATE TIMESTAMP;
ALTER TABLE THRESHOLD_CONFIGURATION ADD DEVIATION_END_DATE TIMESTAMP;

// or, if already applied somewhere
ALTER TABLE THRESHOLD_CONFIGURATION DROP COLUMN TE_REFERENCE;
ALTER TABLE THRESHOLD_CONFIGURATION DROP COLUMN DEVIATION_START_DATE;
ALTER TABLE THRESHOLD_CONFIGURATION DROP COLUMN DEVIATION_END_DATE;

```

Step 3 — Undo original Step 20 – ThresholdConfigurationDto

`mgs-services-interfaces/src/main/java/com/eurocopter/mgs/service/health/common/dto/ThresholdConfigurationDto.java`

Delete these, added after the workcard field.

```

private String teReference;
private Date deviationStartDate;
private Date deviationEndDate;

public final String getTeReference() { return teReference; }
public final void setTeReference(String teReference) { this.teReference = teReference; }
public final Date getDeviationStartDate() { return deviationStartDate; }
public final void setDeviationStartDate(Date d) { this.deviationStartDate = d; }
public final Date getDeviationEndDate() { return deviationEndDate; }
public final void setDeviationEndDate(Date d) { this.deviationEndDate = d; }

```

Remove the `java.util.Date` import too. `serialVersionUID` stays at 1L.

Step 4 — Undo original Step 21 – IndicatorConfigurationDto

`mgs-services-interfaces/src/main/java/com/eurocopter/mgs/service/health/common/dto/IndicatorConfigurationDto.java`

Delete these, added after the `thresholdConfigurationDtos` field.

```

private String filteringLaw;

public final String getFilteringLaw() { return filteringLaw; }
public final void setFilteringLaw(String filteringLaw) { this.filteringLaw = filteringLaw; }

```

Step 5 — Undo original Step 22 – HealthDelegate → updateHealthConfiguration

`mgs-delegate/src/main/java/com/eurocopter/mgs/delegate/health/HealthDelegate.java`

Delete all four lines. The surrounding code returns to exactly what it was.

```
// after: IndicatorConfiguration indicatorConfiguration = indicatorConfigurationsMap.get(...);
indicatorConfiguration.setFilteringLaw(indicatorConfigurationDto.getFilteringLaw());

// after: thresholdConfiguration.setWorkcard(thresholdConfigurationDto.getWorkcard());
thresholdConfiguration.setTeReference(thresholdConfigurationDto.getTeReference());
thresholdConfiguration.setDeviationStartDate(thresholdConfigurationDto.getDeviationStartDate());
thresholdConfiguration.setDeviationEndDate(thresholdConfigurationDto.getDeviationEndDate());
```

This is the important one. The live configuration must stay exactly as the aircraft configuration file defines it. The deviation is a separate record applied on top by session date. Writing deviated values back into the configuration makes the two indistinguishable and breaks acknowledge, because there is nothing left to revert to.

Step 6 — Undo original Step 25 – HealthCommonBusiness → getThresholds

mgs-health-business – the private method near the bottom, called by getIndicatorModels

Delete these four lines, added after setMinimumEngineOilTemperature(...).

```
thresholdModel.setTeReference(existingThresholdConfig.getTeReference());
thresholdModel.setDeviationStartDate(existingThresholdConfig.getDeviationStartDate());
thresholdModel.setDeviationEndDate(existingThresholdConfig.getDeviationEndDate());
thresholdModel.setDeviated(existingThresholdConfig.isDeviationActive());
```

The load-back moves to the client, in Step 14 below, because the deviation no longer lives on the ThresholdConfiguration entity.

Step 7 — Undo original Step 24 – HealthAlarmsPreferenceStore → innerSave

mgs-health-app/src/com/eurocopter/mgs/presentation/configuration/preferences/healthalarms/store/HealthAlarmsPreferenceStore.java

Delete these. innerSave returns to exactly what it was before the BCR – the configuration save no longer knows anything about deviations.

```
indicatorConfigurationDto.setFilteringLaw(wcThresholdModel.getFilteringLawName());

thresholdConfigurationDto.setTeReference(wcThresholdModel.getTeReference());
thresholdConfigurationDto.setDeviationStartDate(
    wcThresholdModel.getDeviationStartDate() != null
    ? wcThresholdModel.getDeviationStartDate()
    : (wcThresholdModel.isDeviated() ? new Date() : null));
thresholdConfigurationDto.setDeviationEndDate(wcThresholdModel.getDeviationEndDate());
```

Remove the java.util.Date import if nothing else in the class uses it. Original Step 23 – the two Date fields on WCThresholdModel – is **kept**: the new design still holds them on the model to display the deviation dates.

Part 2 – Add: the DEVIATION table

Settle this before creating the table. Is a deviation keyed on the indicator, or on indicator plus severity? The Deviation List mock-up shows RED and AMBER as separate rows with different TE references, which means indicator plus severity – and the steps below assume that. But the filtering law is per indicator and the maximum time without acquisition is per component, so both are duplicated across the two rows. This is the one decision that is expensive to reverse.

Step 8 — Deviation – new entity

mgs-jpa-model/src/main/java/com/eurocopter/mgs/datamodel/health/monitoring/Deviation.java

Thirteen deviated parameters plus the keys, the TE and the dates. Active while END_DATE is null – no boolean flag.

```

@Entity
@Table(name = "DEVIATION")
public class Deviation implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @Column(name = "ID", nullable = false)
    private String id;

    @Column(name = "AIRCRAFT_ID", nullable = false)    private String aircraftId;
    @Column(name = "INDICATOR_ID", nullable = false)  private String indicatorId;
    @Column(name = "TE_REFERENCE", nullable = false)  private String teReference;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "START_DATE", nullable = false)    private Date startDate;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "END_DATE")                        private Date endDate;

    // the thirteen deviated parameters
    @Column(name = "SEVERITY")                        private String severity;
    @Column(name = "THRESHOLD_LEVEL")                 private String thresholdLevel;
    @Column(name = "THRESHOLD_TYPE")                 private String thresholdType;
    @Column(name = "THRESHOLD_VALUE")                 private Double value;
    @Column(name = "MIN_VALUE")                       private Double minValue;
    @Column(name = "MAX_VALUE")                       private Double maxValue;
    @Column(name = "LEARNING_TIME")                  private Long learningTime;
    @Column(name = "WORK_CARD")                       private String workcard;
    @Column(name = "MINIMUM_DEVIATION")               private Double minimumDeviation;
    @Column(name = "MEAN_FACTOR")                     private Double meanFactor;
    @Column(name = "DEVIATION_COEFFICIENT")           private Double deviationCoefficient;
    @Column(name = "NB_SUCESSIVE_POINTS")             private Integer nbSuccessivePoints;
    @Column(name = "MAX_TIME_WO_ACQUISITION")         private Long maxTimeWithoutAcquisition;

    @Column(name = "FILTERING_LAW")                  private String filteringLaw;

    public boolean isActive() {
        return endDate == null;
    }
    // constructors and accessors
}

```

Step 9 — DDL / migration script

Create the table and one index. Match the column types to whatever the project uses elsewhere for ids and timestamps.

```

CREATE TABLE DEVIATION (
    ID                                VARCHAR(255) NOT NULL PRIMARY KEY,
    AIRCRAFT_ID                      VARCHAR(255) NOT NULL,
    INDICATOR_ID                     VARCHAR(255) NOT NULL,
    TE_REFERENCE                     VARCHAR(255) NOT NULL,
    START_DATE                       TIMESTAMP    NOT NULL,
    END_DATE                         TIMESTAMP,
    SEVERITY                         VARCHAR(16),
    THRESHOLD_LEVEL                  VARCHAR(16),
    THRESHOLD_TYPE                   VARCHAR(16),
    THRESHOLD_VALUE                  DOUBLE,
    MIN_VALUE                        DOUBLE,
    MAX_VALUE                        DOUBLE,
    LEARNING_TIME                    BIGINT,
    WORK_CARD                        VARCHAR(255),
    MINIMUM_DEVIATION                DOUBLE,
    MEAN_FACTOR                      DOUBLE,
    DEVIATION_COEFFICIENT            DOUBLE,
    NB_SUCESSIVE_POINTS              INTEGER,
    MAX_TIME_WO_ACQUISITION          BIGINT,
    FILTERING_LAW                    VARCHAR(255)
);

CREATE INDEX IDX_DEVIATION_AIRCRAFT ON DEVIATION (AIRCRAFT_ID, END_DATE);

```

Volume is about 300 rows fleet-wide since 2018, so index for correctness rather than throughput. The one index covers both the Deviation List query and the session-date lookup.

Step 10 — DeviationDto – new

mgs-services-interfaces/src/main/java/com/eurocopter/mgs/service/health/common/dto/DeviationDto.java

Same fields as the entity, Serializable, serialVersionUID = 1L. Plain getters and setters. Nothing lazy, so it crosses the wire safely.

Step 11 — DeviationDao – new

mgs-services-impl – same package as HealthDao; copy its package line

Four methods. Follow the @Local @Stateless pattern of HealthDao and inject the same EntityManager.

```
@Local
@Stateless
public class DeviationDao {

    @PersistenceContext
    private EntityManager manager;

    public void create(Deviation deviation) {
        manager.persist(deviation);
    }

    /** REQ 7 and REQ 8 - acknowledge closes the deviation. */
    public void acknowledge(String deviationId, Date endDate) {
        Deviation deviation = manager.find(Deviation.class, deviationId);
        if (deviation != null) {
            deviation.setEndDate(endDate);
        }
    }

    /** Deviation List tab. */
    public List<Deviation> findActive(String aircraftId) {
        return manager.createQuery(
            "SELECT d FROM Deviation d WHERE d.aircraftId = :aircraftId "
            + "AND d.endDate IS NULL", Deviation.class)
            .setParameter("aircraftId", aircraftId)
            .getResultList();
    }

    /** Deviation History tab. */
    public List<Deviation> findHistory(String aircraftId) {
        return manager.createQuery(
            "SELECT d FROM Deviation d WHERE d.aircraftId = :aircraftId "
            + "AND d.endDate IS NOT NULL", Deviation.class)
            .setParameter("aircraftId", aircraftId)
            .getResultList();
    }
}
```

Step 12 — Service and business pass-through

Add the four methods to IHealthCommonService, HealthCommonService, IHealthCommonBusiness and HealthCommonBusiness, converting entity to DeviationDto in the business layer so nothing lazy reaches the client. Follow how getFilteringLawNames was threaded through.

```
List<DeviationDto> findActiveDeviations(String aircraftId);
List<DeviationDto> findDeviationHistory(String aircraftId);
void createDeviations(List<DeviationDto> deviations);
void acknowledgeDeviations(List<String> deviationIds);
```

createDeviations and acknowledgeDeviations take lists because one Apply can produce several, and REQ 7 allows selecting several rows at once.

Step 13 — HealthAlarmsPreferenceStore → the Apply path

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

After isValidData passes, build a DeviationDto for every row where isModifiedAgainstBaseline() is true and send them in one call. Stamp the start date here, once, at creation. This is separate from the existing configuration save – the configuration itself is no longer touched by a deviation.

```

List<DeviationDto> newDeviations = new ArrayList<>();
Date now = new Date();
for (WCThresholdModel model : changedModels) {
    if (!model.isModifiedAgainstBaseline() || model.isDeviated()) {
        continue; // unchanged, or already carries an active deviation
    }
    DeviationDto dto = new DeviationDto();
    dto.setAircraftId(aircraft.getId());
    dto.setIndicatorId(model.getIndicator().getId());
    dto.setSeverity(model.getSeverity().name());
    dto.setTeReference(model.getTeReference());
    dto.setStartDate(now);
    // the thirteen parameters, copied from the model
    ...
    newDeviations.add(dto);
}
if (!newDeviations.isEmpty()) {
    healthBusiness.createDeviations(newDeviations);
}

```

Step 14 — HealthAlarmsPreferenceStore → getThresholdModels

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

Fetch the active deviations once per aircraft, before the component loop, then match by indicator id and severity. This replaces the load-back that used to live in HealthCommonBusiness.getThresholds.

```

// once, in createAllMechanicalComponents
List<DeviationDto> activeDeviations = healthBusiness.findActiveDeviations(aircraft.getId());
Map<String, DeviationDto> byIndicatorAndSeverity = new HashMap<>();
for (DeviationDto dto : activeDeviations) {
    byIndicatorAndSeverity.put(dto.getIndicatorId() + "|" + dto.getSeverity(), dto);
}

// in getThresholdModels, in the loop that already sets the law and the baseline
DeviationDto deviation = byIndicatorAndSeverity.get(
    indicator.getId() + "|" + thresholdModel.getSeverity().name());
if (deviation != null) {
    thresholdModel.setDeviated(true);
    thresholdModel.setTeReference(deviation.getTeReference());
    thresholdModel.setDeviationStartDate(deviation.getStartDate());
    // apply the deviated parameters over the configuration values
    thresholdModel.setDefaultValue(deviation.getValue());
    thresholdModel.setMinValue(deviation.getMinValue());
    thresholdModel.setFilteringLawName(deviation.getFilteringLaw());
    ...
}

```

Set the baseline *before* this block, from the configuration values, and overwrite the model afterwards with the deviated ones. The baseline then holds the configuration and the model holds what the user sees, which is exactly what acknowledge needs to revert to.

What this makes easier

Requirement	Now becomes
REQ 5, 6 – Deviation List	findActive(aircraftId). No diffing, no joining back to threshold configurations.
REQ 11 – Deviation History	findHistory(aircraftId), with START_DATE and END_DATE giving the range directly.
REQ 8 – Acknowledge	Set END_DATE. The indicator reverts to the configuration automatically, because the configuration was never changed.
REQ 12 – Applicability	One WHERE clause: START_DATE < sessionDataDate AND (END_DATE IS NULL OR END_DATE >= sessionDataDate).
REQ 10 – Config update	Acknowledge every active row for the SN in one update.
REQ 13 – DACOM	GSC_DEVIATIONS.sql is a single SELECT over this table.

The estimate does not move. Packages 4 and 5 change shape rather than size, and packages 7, 8, 10 and 13 each get slightly cheaper because the queries are now direct.